



Master 1 (M1)

*Département d'informatique
l'UFR-SEA/
UJKZ*

Bases de Données Avancées

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ*

Janvier 2023

<http://kisitokab.xyz>



- **Prérequis**
 - Principes des systèmes de base de données
- **Volume horaire**
 - 30 heures
 - 3 Crédits
- **Méthode d'évaluation**
 - 1 Devoir
 - 1 Projet



Justification du Cours/ module



Résultats de l'apprentissage



Contenu-sujets du Cours

- Introduction à la conception des BdD
- Introduction à la réalisation des BdD
- Introduction à l'administration des BdD
- Les BdD réparties
- Les BdD Objets
- Les extensions XML



Master 1 (M1)

*Département d'informatique
l'UFR-SEA/
UJKZ*

CHAPITRE 2

Introduction à la réalisation des Bases de Données

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'Informatique
à l'UFR-SEA / Université JKZ*

Janvier 2021

<http://kisitokab.xyz>

SQL:

- Langage de Définition de données
- Langage de Manipulation de données

Objectifs

- „ Utilité et importance de SQL,
- „ Types de données supportés par le standard SQL.
- „ Utilité de la propriété d'amélioration d'intégrité de SQL.
- „ Comment définir des contraintes d'intégrité avec SQL.
- „ Comment utiliser la gestion d'intégrité dans les requêtes
CREATE et ALTER TABLE.
- „ Utilité des vues.
- „ Comment créer et effacer les vues avec SQL.
- „ Comment le SGBD exécute les opérations sur les vues.
- „ Sous quelles conditions les vues sont modifiables.
- „ Avantages et désavantages des vues.

Objectifs de SQL

- „ Idéalement, un langage de base de données devrait permettre à l'utilisateur de:
 - ... Créer la structure de la base de données et de ses tables,
 - ... Faire des insertion, modification, suppression de données des tables,
 - ... Faire des requêtes simples ou complexes,
- „ L'utilisateur doit pouvoir réaliser ces tâches avec le minimum d'effort et la structure et la syntaxe de commande doivent être relativement facile à apprendre,
- „ Il doit être suffisamment portable.

Objectifs de SQL

- „ SQL est un langage orienté-transformation avec 2 composants majeurs:
 - ... Le DDL pour définir la structure de la base de données,
 - ... Le DML pour récupérer et mettre à jour les données,
- „ Jusqu'à SQL3, SQL ne contenait pas de commande de contrôle de flot d'instruction. C'était alors implanté en utilisant un langage de programmation ou interactivement en fonction des décisions de l'utilisateur.

Objectifs de SQL

„ SQL est relativement facile à apprendre:

... Il est non procédural - vous spécifiez ce que vous voulez et non comment vous le voulez,

... Il est essentiellement sans format. C'est-à-dire que le format n'a pas d'influence sur la compilation,

... Consiste en des mots communs en anglais:

```
SELECT staffNo, IName, salary
```

```
FROM Staff
```

```
WHERE salary > 10000;
```

Objectifs de SQL

- „ Peut être utilisé par différents types d'utilisateur comme les DBA, gestionnaires, développeurs d'application et autres,
- „ Un standard ISO existe maintenant pour SQL, ce qui fait de lui un langage standard formel et de facto pour les bases de données relationnelles.

Histoire de SQL

- „ En 1974, D. Chamberlin (IBM San Jose Laboratory) a défini un langage appelé 'Structured English Query Language' (SEQUEL),
- „ Une version révisée, SEQUEL/2, est définie en 1976 mais le nom a été changé pour SQL pour des raisons légales,
- „ En anglais, il est encore prononcé 'si-quel', même si la prononciation officielle est 'S-Q-L',
- „ IBM a ensuite produit un prototype de SGBD appelé System R, basé sur SEQUEL/2.

Histoire de SQL

- „ À la fin des années 70, ORACLE est apparu et a été probablement le premier SGBDr commercial basé sur SQL,
- „ En 1987, ANSI et ISO ont publiés un standard initial pour SQL,
- „ En 1989, ISO a publié un addendum qui définit une fonction d'amélioration d'intégrité,
- „ En 1992, la première révision majeure au standard ISO a été réalisée, appelée SQL2 ou SQL/92,
- „ En 1999, SQL3 a été présenté. Il contient un support pour la gestion de données Orienté-Objet.

SQL : Définition des données (DDL)

Écriture de commandes SQL

- „ La requête SQL est constituée de mots **réservés** et de mots définis par l'utilisateur,
- „ Les mots **réservés** sont une partie fixe de SQL et doivent être épelés exactement comme définis et ne peuvent être **divisés** sur 2 lignes,
- „ Les mots **définis** par l'utilisateur représentent les noms des différents objets de la base de données tels que les tables, les colonnes et les vues,
- „ La majorité des composants d'une requête SQL sont **insensibles** à la casse,
- „ Plus facile à lire avec des **indentations** et la séparation en ligne:

Valeurs littérales

- „ Les valeurs littérales sont des constantes utilisées dans les requêtes SQL,
- „ Toutes les valeurs littérales non numériques doivent être encadrés de guillemets simples (par exemple: 'Londres'),
- „ Tous les littéraux numériques doivent ne pas être encadrés (par exemple: 650.00).

Définition de données

- „ Le DDL SQL permet aux objets de base de données tels que les schémas, les domaines, les tables, les vues et les indexes d'être créés et détruits,
- „ Les principales requêtes DDL SQL sont:

CREATE/ALTER TABLE

CREATE VIEW

DROP TABLE

DROP VIEW;

ISO SQL Data Types

Tableau 6.1 Types de données ISO de SQL.

Type de donnée	Déclarations			
booléen	BOOLEAN			
caractère	CHAR	VARCHAR		
bit [†]	BIT	BIT VARYING		
numérique exact	NUMERIC	DECIMAL	INTEGER	SMALLINT
numérique approximatif	FLOAT	REAL	DOUBLE PRECISION	
date-heure	DATE	TIME	TIMESTAMP	
intervalle	INTERVAL			
grand objet	CHARACTER LARGE OBJECT		BINARY LARGE OBJECT	

[†] Les types BIT et BIT VARYING ont été supprimés dans la norme SQL:2003.

„ Confer documentation Oracle.

Propriété d'amélioration d'intégrité

„ Considérons 5 types de contraintes d'intégrité:

... Donnée requises (**NOT NULL**) . **Exemple :**,

position VARCHAR(10) NOT NULL

... Contraintes de domaine . **Exemple :**,

sexe CHAR NOT NULL CHECK (sexe IN ('H', 'F'))

... Intégrité d'entité,

... Intégrité référentielle,

... Contraintes d'entreprise.

PAI - Intégrité d'entité

- „ La clé primaire d'une table doit contenir une valeur unique non-nulle pour chaque rangée,
- „ Le standard ISO supporte la clause **PRIMARY KEY** dans les requêtes **CREATE** et **ALTER TABLE**. **Exemple** :

PRIMARY KEY(numPersonnel)

PRIMARY KEY(numClient, numPropriété)

- „ Il ne peut y avoir qu'une clause PRIMARY KEY par table,
- „ On peut aussi assurer l'unicité d'une clé alternative par le **UNIQUE**. **Exemple** :

UNIQUE(numTelephone)

PAI - Intégrité référentielle

- „ **FK** est une colonne ou un ensemble de colonnes qui relie chaque rangée de la table enfant contenant la FK à la rangée de la table parent contenant la PK,
- „ L'intégrité référentielle signifie que, si la FK contient une valeur, cette valeur doit référer à une rangée existante de la table parent,
- „ Le standard ISO supporte la définition des FK avec **FOREIGN KEY** dans CREATE et ALTER TABLE. **Exemple :**

FOREIGN KEY (numFiliale) REFERENCES Filiale

PAI - Intégrité référentielle

- „ Tout INSERT/UPDATE qui essaie de créer une valeur de FK dans la table enfant sans correspondance de valeur de clé primaire dans le parent est rejeté,
- „ Une action prise qui tente de mettre à jour/effacer une valeur de clé primaire de la table parent avec une valeur correspondante dans la table enfant est dépendant de l'action référentielle spécifiée par les clauses **ON**

UPDATE et ON DELETE :

- **CASCADE**
- **SET NULL**
- **SET DEFAULT**
- **NO ACTION**

PAI - Intégrité référentielle

- „ **CASCADE**: efface les rangés de l'enfant si la rangée du parent est effacé, ainsi de suite en cascade,
- „ **SET NULL**: Lors de l'effacement dans la table parent, la valeur de la colonne FK de l'enfant est mis à NULL.
Seulement si la colonne FK est définie NULL,
- „ **SET DEFAULT**: Lors de l'effacement du parent chaque valeur de la colonne FK de l'enfant est changée pour sa valeur spécifiée par défaut. Valide seulement si DEFAULT est spécifié sur la colonne FK,
- „ **NO ACTION**: Empêche la modification / effacement du parent. C'est l'action par défaut.

PAI - Intégrité référentielle

Exemples :

FOREIGN KEY (numPersonnel) REFERENCES Personnel
ON DELETE SET NULL

FOREIGN KEY (numPropriétaire) REFERENCES
Propriétaire ON UPDATE CASCADE

”

CREATE TABLE

CREATE TABLE NomTable

{(nomColonne TypeDonnee [NOT NULL] [UNIQUE]

[DEFAULT ValeurParDefaut]

[CHECK Condition] [...]}

[PRIMARY KEY (ListeDeColonnes),]

{[UNIQUE (ListeDeColonnes),] [...]}

{[FOREIGN KEY (ListeDeColonnes)

REFERENCES NomTableParent [(ListeDeColonnes)],

[ON UPDATE ActionReferentielle]

[ON DELETE ActionReferentielle]] [...]}

{[CHECK (Condition)] [...]} }

CREATE TABLE

- „ Créé une table avec une ou plusieurs colonnes du type de donnée spécifié,
- „ Avec NOT NULL, le système rejette toute tentative d'insérer une valeur nulle dans la colonne,
- „ Peut spécifier une valeur par DEFAULT pour une colonne,
- „ La clause FOREIGN KEY spécifie les FK ainsi que les Actions référentielles.

Exemple 6.1 - CREATE TABLE

```
CREATE TABLE PropriétéÀLouer (numPropriété
    VARCHAR(5) NOT NULL,
    pièces SMALLINT CHECK(pièces BETWEEN 1 AND 15)
    NOT NULL DEFAULT 4,
    location DECIMAL(6,2)CHECK(location BETWEEN 0
    AND 9999.99)    NOT NULL DEFAULT 600,
    numPropriétaire VARCHAR(5) NOT NULL,
    numPersonnel VARCHAR(5),
    numFiliale SMALLINT NOT NULL,
    PRIMARY KEY (numPropriété),
    FOREIGN KEY (numPersonnel) REFERENCES
    Personnel ON DELETE SET NULL ON UPDATE
    CASCADE ...);
```

Exemple 6.2 (b) - ALTER TABLE

- „ Ajoute ou efface une colonne à une table,
- „ Ajoute ou efface une contrainte à une table,
- „ Exemples
 - Enlever la contrainte sur la limite de gestion de PropriétéÀLouer:
ALTER TABLE PropriétéÀLouer
DROP CONSTRAINT PersonnelNeGèrePasTrop ;
 - Ajouter la colonne nbrePiècesFavori à la table Client.
ALTER TABLE Client
ADD nbrePiècesFavori integer CHECK
(nbrePiècesFavori BETWEEN 1 AND 15) ;

DROP TABLE

„ **DROP TABLE** nomTable [**RESTRICT** | **CASCADE**]

Exemple : **DROP TABLE** PropriétéÀLouer;

Efface la table nommée et tout son contenu,

- „ Avec **RESTRICT**, si un objet dépend de la table effacée, SQL ne permet pas l'effacement.
- „ Avec **CASCADE**, SQL efface tous les objets dépendants (et les objets qui dépendent de ces objets).

SQL : Manipulation des données (DML)

Manipulation des données

Requête **SELECT**

SELECT [DISTINCT | ALL]

{* | [expressionDeColonne [**AS** nouveauNom]] [,...]

FROM nomTable [alias] [, ...]

[**WHERE** condition]

[**GROUP BY** listeDeColonne] [**HAVING** condition]

[**ORDER BY** listeDeColonne]

Requête SELECT

FROM Spécifie la (ou les) table(s) à utiliser,

WHERE Filtre les rangées,

GROUP BY Forme des groupes de rangées
avec la même valeur de colonne,

HAVING Filtre les groupes,

SELECT Spécifie les colonnes affichées
dans la réponse,

ORDER BY Spécifie l'ordre d'affichage,

„ l'ordre des clauses dans la requête ne peut être
changé,

„ Seuls **SELECT** et **FROM** sont **obligatoires**.

Exemple 5.1 Toutes les colonnes, toutes les lignes

- „ Énumérer les détails complets de tous les membres du personnel:

```
SELECT numPersonnel, prénom, nom, fonction,  
        genre, dateDeNaissance, salaire, numFiliale  
  
FROM Personnel;
```

- „ On peut utiliser l'astérisque ***** comme abréviation de 'toutes les colonnes':

```
SELECT *  
  
FROM Personnel;
```



Exemple 5.1 Toutes les colonnes, toutes les lignes

numPersonnel	prénom	nom	fonction	genre	dateDeNaissance	salaire	numFiliale
EP21	Jean	Blanc	Gérant	M	1 Oct 45	30000.00	F005
EM37	Anne	Bertrand	Assistant	F	10 Nov 60	12000.00	F003
EM14	David	Renault	Superviseur	M	24 Mar 58	18000.00	F003
EG9	Marie	Thale	Assistant	F	19 Fév 70	9000.00	F007
EM5	Suzanne	Brière	Gérant	F	3 Jun 40	24000.00	F003
EP41	Julie	Hette	Assistant	F	13 Jun 65	9000.00	F005

Exemple 5.2 Colonnes spécifiques, toutes les lignes

„ *Produire une liste de tous les membres du personnel, ne montrant que le numéro, le prénom, le nom et le salaire des employés:*

```
SELECT numPersonnel, prénom, nom, salaire  
FROM Personnel ;
```


Exemple 5.2 Colonnes spécifiques,
toutes les lignes

numPersonnel	prénom	nom	salaire
EP21	Jean	Blanc	30000.00
EM37	Anne	Bertrand	12000.00
EM14	David	Renault	18000.00
EG9	Marie	Thale	9000.00
EM5	Suzanne	Brière	24000.00
EP41	Julie	Hette	9000.00

Exemple 5.3 DISTINCT

„ Énumérer les numéros de toutes les propriétés
à louer qui ont été déjà visitées:

```
SELECT numPropriété  
FROM Visite;
```

„ On utilise DISTINCT pour éliminer les
doublons:

```
SELECT DISTINCT numPropriété  
FROM Visite;
```

numPropriété
LG14
LM4
LM4
LG14
LM36

numPropriété
LG14
LM4
LM36

Exemple 5.4 Champs calculés

„ Produire une liste de tous les salaires mensuels des membres du personnel, montrant les numéros de membre du personnel, les prénoms, les noms et le salaire mensuel:

```
SELECT numPersonnel, prénom, nom, salaire/12
```

```
FROM Personnel;
```

„ Pour renommer une colonne, utiliser la clause AS:

```
SELECT numPersonnel, prénom, nom,
```

```
salaire/12 AS salaireMensuel
```

```
FROM Personnel;
```

numPersonnel	prénom	nom	col4
EP21	Jean	Blanc	2500.00
EM37	Anne	Bertrand	1000.00
EM14	David	Renault	1500.00
EG9	Marie	Thale	750.00
EM5	Suzanne	Brière	2000.00
EP41	Julie	Héne	750.00

Dr Kiswendsida Kisito Kaboré.

<http://Kisitokab.xyz>

Exemple 5.5 Une condition de recherche de type comparaison

„ Énumérer les membres du personnel dont le salaire est supérieur à 10 000 €:

```
SELECT numPersonnel, prénom, nom, fonction, salaire  
FROM Personnel  
WHERE salaire > 10000;
```

numPersonnel	prénom	nom	fonction	salaire
EP21	Jean	Blanc	Gérant	30000.00
EM37	Anne	Bertrand	Assistant	12000.00
EM14	David	Renaud	Superviseur	18000.00
EM5	Susanne	Brière	Gérant	24000.00

Opérateurs de comparaison

- „ = **égal**, <> n'est pas égal à (norme ISO), != n'est pas égal à (autorisé dans certains dialectes),
- „ < est plus petit que, <= est **inférieur** ou égal à,
- „ > est plus grand que, >= est **supérieur** ou égal à,
- „ **Une expression s'évalue de gauche à droite,**
- „ **Les sous-expressions entre parenthèses sont évaluées en premier lieu,**
- „ **NOT** est évalué avant **AND** et **OR**,
- „ **AND** est évalué avant **OR**,
- „ L'usage des **parenthèses** est toujours recommandé, de manière à **éliminer toute ambiguïté.**

Exemple 5.6 Critère de recherche de comparaison combiné

„ Énumérer les adresses de toutes les filiales de Paris et
de Montréal:

SELECT *

FROM Filiale

WHERE ville = 'Paris' OR ville = 'Montréal';

numFiliale	rue	ville	codePostal
F005	67 Champs-Élysées	Paris	75000
F003	243 rue des Roses	Montréal	H2L 5Y7
F002	59 Bd St-Germain	Paris	75018

Exemple 5.7 Critère de recherche d'écart

„ Énumérer les membres du personnel dont le salaire est compris entre 20 000 € et 30 000 €:

SELECT numPersonnel, prénom, nom, fonction, salaire

FROM Personnel

WHERE salaire **BETWEEN** 20000 **AND** 30000

Le test **BETWEEN** est inclusif,

Il y a aussi la négation **NOT BETWEEN**,

numPersonnel	prénom	nom	fonction	salaire
EP21	Jean	Blanc	Gérant	30000.00
EM5	Suzanne	Brière	Gérant	24000.00

Exemple 5.8 Appartenance à un ensemble

„ Énumérer tous les gérants et les superviseurs parmi les membres du personnel:

```
SELECT numPersonnel, prénom, nom, fonction  
FROM Personnel  
WHERE fonction IN ('Gérant', 'Superviseur');
```

numPersonnel	prénom	nom	fonction
EP21	Jean	Blanc	Gérant
EM14	David	Renault	Superviseur
EM5	Suzanne	Brière	Gérant

Exemple 5.8 Appartenance à un ensemble

- „ Il y a aussi la négation (NOT IN),
- „ IN n'ajoute pas tellement à l'expressivité de SQL. On peut aussi écrire:

```
SELECT numPersonnel, prénom, nom, fonction
FROM Personnel
WHERE fonction = 'Gérant' OR
fonction = 'Superviseur' ;
```

- „ IN est efficace lorsque l'ensemble contient plusieurs valeurs.

Exemple 5.9 Correspondance à un masque

„ Retrouver tous les propriétaires qui portent la chaîne
'Montréal' dans leur adresse:

```
SELECT numPropriétaire, prénom, nom, adresse, numTél  
FROM PropriétairePrivé  
WHERE adresse like '%Montréal% ';
```

numPropriétaire	prénom	nom	adresse	numTél
CP87	Claude	Vézina	2115 de la Gauchetière, Montréal H2L 5Y7	+1-(514) 656-7891
CP40	Brigitte	Gauthier	18 rue Edison, Montréal H2L 5Y7	+1-(418) 871-9635
CP93	Guy	Lacasse	888 boul. Laviolette, Montréal H2L 5Y7	+1-(819) 524-1075

Exemple 5.9 Correspondance à un masque

„ SQL possède 2 symboles de correspondance à un masque :

... %: séquence de zéro ou plusieurs caractères,

... _ (sous-ligné): un seul caractère, n'importe lequel,

„ LIKE '%Montréal%' signifie une séquence de n'importe quelle longueur contenant 'Montréal'.

Exemple 5.10 Condition de recherche de NULL

„ Énumérer les détails des visites de la propriété à louer LM4 qui n'ont pas encore reçu de commentaire:

„ Il faut tester le null explicitement à l'aide des mots clés IS NULL:

```
SELECT numClient, DateVisite  
FROM Visite  
WHERE numPropriété='LM4' AND  
commentaire IS NULL ;
```




Exemple 5.10 Condition de recherche de NULL

Tableau 5.10 Table obtenue en résultat de l'exemple 5.10.

numClient	dateVisite
CL56	26 Mai 04

„ La négation (IS NOT NULL) peut tester pour les valeurs non nulles.

Exemple 5.11 Tri selon une seule colonne

„ Produire une liste de tous les salaires des membres du personnel, classés par ordre décroissant de salaires:

```
SELECT numPersonnel, prénom, nom, salaire  
FROM Personnel  
ORDER BY salaire DESC;
```

numPersonnel	prénom	nom	salaire
EP21	Jean	Blanc	30000.00
EM5	Suzanne	Brière	24000.00
EM14	David	Renault	18000.00
EM37	Anne	Bertrand	12000.00
EG9	Marie	Thale	9000.00
EP41	Julie	Hette	9000.00

Exemple 5.12 Classement selon plusieurs colonnes

„ Produire une liste abrégée des propriétés, classée par ordre de types de biens immobiliers:

```
SELECT numPropriété, type, pièces, location  
FROM PropriétéALouer  
ORDER BY type;
```

numPropriété	type	pièces	location
LP94	Appartement	4	400
LM4	Appartement	3	350
LM36	Appartement	3	375
LM16	Appartement	4	450
LG14	Maison	6	650
LM21	Maison	5	600

Exemple 5.12 Classement selon plusieurs colonnes

„ Quatre propriétés de type Appartement dans cette liste -
puisque aucun champ de tri secondaire n'a été spécifié,
le système présente ces lignes dans un ordre
quelconque,

„ Pour classer en ordre de location, on spécifie un ordre
secondaire:

```
SELECT numPropriété, type,  
       pièces, location  
FROM PropriétéALouer  
ORDER BY type ASC,
```

numPropriété	type	pièces	location
LM16	Appartement	4	450
LP94	Appartement	4	400
LM36	Appartement	3	375
LM4	Appartement	3	350
LG14	Maison	6	650
LM21	Maison	5	600

```
location DESC ;
```

AJOUT DE TUPLES DANS UNE RELATION

INSERT

INSERT INTO NomDeTable [(listeDeColonnes)]

VALUES (listeDeValeursDeDonnées);

- „ **listeColonnes** est optionnel; si elle est omise, SQL assume que tous les champs de la table sont utilisés dans l'ordre de création,
- „ Toute colonne omise doit permettre les **NULL**, à moins qu'une valeur ait été spécifiée par **DEFAULT**,
- „ Le nombre d'items dans chaque liste doit être le même,
- „ Il doit y avoir correspondance directe dans la position des items des deux listes,
- „ Le type de données de chaque item de **listeDeValeurs** doit être compatible avec le type de données avec les colonnes correspondantes.

Exemples 5.35 et 5.36 **INSERT ... VALUES**

„ *Insérer une nouvelle ligne dans la table Personnel et
donner des valeurs à toutes les colonnes:*

```
INSERT INTO Personnel  
VALUES ('EM16', 'Alain', 'Leblanc', 'Assistant', 'M',  
DATE '1957-05-25', 8300, 'F003');
```

„ *Insérer une nouvelle ligne dans la table Personnel et
donner des valeurs à toutes les colonnes obligatoires :*

```
INSERT INTO Personnel (numPersonnel, prénom,  
nom, fonction, salaire, numFiliale)  
VALUES ('SM44', 'Anne', 'Jeamme', 'Assistant', 8100,  
'F003');
```

Exemple 5.37 INSERT ... SELECT

- „ Seconde forme d'INSERT qui permet à plusieurs rangées d'être copiées à partir de une ou plusieurs tables vers une autre:

```
INSERT INTO TableName [ (listeColonne) ]
```

```
SELECT ...
```

- „ Supposons une table `PersonneComptePropriété` qui contient les noms des employés et le nombre des propriétés qu'ils gèrent:

```
PersonneComptePropriété(numPersonnel, Prénom, Nom, nbProp)
```

- „ remplir `PersonneComptePropriété` en utilisant les tables `Personnel` et `PropriétéALouer`.

Exemple 5.37 INSERT ... SELECT

```
INSERT INTO PersoComptePropriété (SELECT
    p.numPersonnel, prénom, nom, COUNT(*)
FROM Personnel p, PropriétéALouer l WHERE
    p.numPersonnel = l.numPersonnel GROUP BY
    p.numPersonnel, prénom, nom)
UNION
(SELECT numPersonnel, prénom, nom, 0 FROM
    Personnel WHERE numPersonnel NOT EXISTS
    (SELECT * FROM PropriétéALouer l WHERE
        l.numPersonnel = p.numPersonnel));
```

- „ Si la seconde partie de l'UNION est omise, cela exclue les employés qui en ce moment ne gèrent aucune propriété.

UPDATE

UPDATE NomTable

SET nomColonne1 = Valeur1

[, nomColonne2 = Valeur2...]

[WHERE ConditionDeRecherche]

- „ **NomTable** un nom de table ou une vue modifiable,
- „ **SET** spécifie les noms de une ou plusieurs colonnes qui sont à mettre à jour,
- „ **WHERE** est optionnel:
 - ... Si spécifié, seules les rangées qui satisfont ConditionDeRecherche sont mise à jour,
- „ Les nouvelles valeurs doivent être compatibles avec le type de données de colonnes correspondantes.

Exemple 5.38-5.40 UPDATE

- „ Donnez à tous les employés 3% d'augmentation:

```
UPDATE Personnel
```

```
SET salaire = salaire * 1.03;
```

- „ Donnez au gérant 5% d'augmentation:

```
UPDATE Personnel
```

```
SET salaire = salaire * 1.05
```

```
WHERE fonction = 'Gérant';
```

- „ Donner une promotion à David Ford (numPersonnel = 'EM14') devenant gérant et changer son salaire pour 18,000\$:

```
UPDATE Personnel
```

```
SET fonction = 'Gérant', salaire = 18000
```

```
WHERE numPersonnel = 'EM14';
```

DELETE

DELETE FROM NomTable

[WHERE ConditionDeRecherche]

- „ NomTable peut être un nom de table ou une vue modifiable,
- „ ConditionDeRecherche est optionnelle; si omise, Toutes les rangées de la table sont effacées. DELETE n'efface pas la table,
- „ Effacer toutes les visites de la propriété LM4:

```
DELETE FROM Visite
```

```
WHERE numPropriété = 'LM4';
```

- „ Effacer tous les enregistrements de la table des visites:

```
DELETE FROM Visite;
```

Requête SELECT - Agrégation

Requête SELECT - Agrégation

„ Le standard ISO définit cinq fonctions d'agrégation:

... **COUNT** retourne le nombre de valeurs de la colonne spécifiée,

... **SUM** retourne la somme des valeurs de la colonne spécifiée,

... **AVG** retourne la moyenne des valeurs de la colonne spécifiée,

... **MIN** retourne la petite valeur de la colonne spécifiée,

... **MAX** retourne la grande valeur de la colonne spécifiée.

Requête SELECT - Agrégation

- „ Chacune s'exécute sur une seule colonne d'une table et retourne une valeur,
- „ **COUNT, MIN, et MAX** s'applique à des champs numériques et non numériques, mais SUM et AVG peuvent être utilisés sur des champs numériques seulement,
- „ Outre **COUNT(*)**, chaque fonction élimine les valeurs nulles en premier et s'exécute seulement sur les valeurs restantes.

Requête SELECT - Agrégation

- „ **COUNT(*)** compte toutes les rangées de la table, sans égard aux valeurs nulles ou au doublons,
- „ On peut utiliser **DISTINCT** avant le nom des colonnes pour éliminer les doublons,
- „ **DISTINCT** n'a pas d'effet sur **MIN** / **MAX**, mais peut en avoir sur SUM/AVG,
- „ Les fonctions d'agrégation peuvent être utilisées seulement dans la liste des champs d'un SELECT et dans la clause **HAVING**,
- „ Si la liste du SELECT inclut une fonction d'agrégation et qu'il n'y a pas de clause **GROUP BY**, la liste SELECT ne peut référencer une colonne sans fonction d'agrégation.

Agrégation et Exemple 5.13

COUNT(*)

„ Par exemple, ce qui suit est interdit :

```
SELECT numPersonnel, COUNT(salaire)
```

```
FROM Personnel ;
```

„ *Quel est le nombre de biens immobiliers dont le montant de location mensuel est supérieur à 350 €?*

```
SELECT COUNT(*) AS nombre
```

```
FROM PropriétéALouer
```

```
WHERE location > 350 ;
```

nombre
5

Exemple 5.14 **COUNT(DISTINCT)**

„ *Quel est le nombre de biens immobiliers différents qui ont été visités en mai 2004 ?*

```
SELECT COUNT(DISTINCT numPropriété) AS nombre  
FROM visite  
WHERE dateVisite BETWEEN '01 Mai 04' AND '31 Mai 04';
```

Exemple 5.15 **COUNT** et **SUM**

„ Compter le nombre total de gérants et calculer la somme
de leurs salaires.

```
SELECT COUNT(numPersonnel) AS nombre,  
       SUM(salaire) AS somme  
FROM Personnel  
WHERE fonction = 'Gérant' ;
```

nombre	somme
2	54000.00

Exemple 5.16 **MIN, MAX, AVG**

„ Calculer les salaires minimum, maximum et moyen de tous les membres du personnel:

```
SELECT MIN (salaire) AS mini, MAX(salaire) AS maxi,  
       AVG(salaire) AS moyenne  
FROM Personnel ;
```

mini	maxi	moyenne
9000.00	30000.00	17000.00

Requête SELECT - regroupement

- „ On utilise la clause GROUP BY pour obtenir des
Sous-totaux,
- „ SELECT et GROUP BY intègrent chaque item de la liste
SELECT qui doit être une valeur unique par groupe, et la
clause SELECT peut contenir seulement:
 - ... Nom de colonnes,
 - ... Fonctions d'agrégation,
 - ... Constantes,
 - ... Expression combinant les précédents.

Requête SELECT - regroupement

- „ Tous les noms de colonne de la liste du SELECT doivent apparaître dans la clause GROUP BY à moins que ce nom soit dans une fonction d'agrégation,
- „ Si WHERE est utilisé avec GROUP BY, WHERE est appliqué en premier, ensuite les groupes sont formés avec les rangées satisfaisant le prédicat,
- „ ISO considère les valeurs nulles comme équivalentes pour le GROUP BY.

Exemple 5.17 **GROUP BY**

„ Trouver le nombre d'employés qui travaillent dans
chaque filiale et calculer, pour chaque filiale, la somme
de leurs salaires:

```
SELECT numFiliale, COUNT(numPersonnel) AS nombre,  
       SUM(salaire) AS somme  
FROM Personnel  
GROUP BY numFiliale  
ORDER BY numFiliale ;
```

numFiliale	nombre	somme
F003	3	54000.00
F005	2	39000.00
F007	1	9000.00

Regroupement restreint - clause HAVING

- „ La clause HAVING est conçue pour être utilisée avec GROUP BY pour restreindre les groupes qui apparaissent dans le résultat,
- „ Similaire à WHERE, mais WHERE filtre les lignes individuellement tandis que HAVING filtre les groupes,
- „ Les noms de colonnes dans la clause HAVING doivent aussi apparaître dans la liste du GROUP BY ou être contenus dans une fonction d'agrégation.

Exemple 5.18 **HAVING**

„ Pour toutes les filiales de plus d'un employé, trouver le
nombre de personnes travaillant dans chaque filiale et la
somme de leurs salaires:

```
SELECT numFiliale, COUNT(numPersonnel) AS nombre,  
       SUM(salaire) AS somme  
FROM Personnel  
GROUP BY numFiliale  
HAVING COUNT(numPersonnel)>1  
ORDER BY numFiliale ;
```

numFiliale	nombre	somme
F003	3	54000.00
F005	2	39000.00

SOUS-REQUÊTES

Sous-requêtes

- „ Certaines requêtes SQL peuvent avoir un SELECT inclus à l'intérieur,
- „ Une sous-sélection peut être utilisée dans un WHERE ou un FROM d'un SELECT externe, où on l'appelle sous-requête ou requête imbriquée,
- „ Les sous-sélections peuvent aussi apparaître dans un INSERT, UPDATE ou un DELETE.

Exemple 5.19 **Sous-requête avec égalité**

„ Énumérer tous les employés qui travaillent à la filiale
sise '243 rue des Roses':

```
SELECT numPersonnel, prénom, nom, fonction
FROM Personnel
WHERE numFiliale=
    (SELECT numFiliale
     FROM Filiale
     WHERE rue = ' 243 rue des Roses');
```

Exemple 5.19 **Sous-requête avec égalité**

- „ Le SELECT interne trouve le numéro de succursale pour la succursale à '243 rue des roses' ('F003'),
- „ Le SELECT externe retrouve alors les détails de tous les employés de cette succursale,
- „ Le SELECT externe s'exécute alors comme suit:

```
SELECT numPersonnel, prénom, nom, fonction  
FROM Personnel  
WHERE numfiliale = 'F003';
```

numPersonnel	prénom	nom	fonction
EM37	Anne	Bertrand	Assistant
EM14	David	Renault	Superviseur
EM5	Suzanne	Brière	Gérant

Exemple 5.20 **Sous-requête avec agrégation**

„ Énumérer tous les employés dont le salaire est supérieur au salaire moyen et montrer où se positionne leur salaire par rapport au salaire moyen:

```
SELECT numPersonnel, prénom, nom,  
       salaire - (SELECT AVG(salaire) FROM Personnel) AS  
           différenceSalaire  
FROM Personnel  
WHERE salaire > (SELECT AVG(salaire) FROM  
Personnel);
```

Exemple 5.20 Sous-requête avec agrégation

- „ Impossible: 'WHERE salaire > AVG(salaire)',
- „ Plutôt, utiliser une sous-requête pour trouver la moyenne de salaire (17000), ensuite se servir du SELECT externe pour trouver les employés donc le salaire est plus grand:

```
SELECT numPersonnel, prénom, nom, salaire -17000  
      AS différenceSalaire  
FROM Personnel  
WHERE salaire >17000;
```

numPersonnel	prénom	nom	fonction	salaire
EP21	Jean	Blanc	Gérant	13000.00
EM14	David	Renaud	Superviseur	1000.00
EM5	Suzanne	Brière	Gérant	7000.00

Règles de sous-requête

- „ ORDER BY ne peut être utilisé dans une sous-requête (bien que ce soit possible dans le SELECT externe),
- „ Dans un WHERE, une liste de sous-requête SELECT doit consister en un seul nom de colonne ou une seule expression, sauf pour les sous-requêtes avec EXISTS ou celles incluses dans le FROM,
- „ Par défaut, les noms de colonne réfèrent au nom de la table dans le FROM de la sous-requête,
- „ Lorsqu'une sous-requête est un opérande d'une comparaison, la sous-requête doit apparaître du côté droit,

Exemple 5.21 **sous-requête: IN**

„ Énumérer les propriétés à louer gérées par les employés
de la filiale sise '243 rue des Roses':

```
SELECT numPropriété, rue, ville, codePostal, type, pièces,  
location FROM PropriétéALouer  
WHERE numPersonnel IN (SELECT numPersonnel  
FROM Personnel  
WHERE numFiliale = (SELECT numFiliale  
FROM Filiale  
WHERE rue = '243 rue des Roses'));
```

numPropriété	rue	ville	codePostal	type	pièces	location
LM16	911 de l'Aéroport #1809	Montréal	H2L 5Y7	Appartement	4	450
LM36	1780 boul. Central #210	Montréal	H2L 5Y7	Appartement	3	375
LM21	45 chemin du Lac	Montréal	H2L 5Y7	Maison	5	600

ANY et ALL

- „ **ANY** et **ALL** peuvent être utilisés avec des sous-requêtes qui produisent une colonne simple de plusieurs valeurs,
- „ Avec **ALL**, la condition sera vraie seulement si elle est satisfaite par toutes les valeurs produites par la sous-requête,
- „ Avec **ANY**, la condition sera vraie si elle est satisfaite par une quelconque valeur produite par la sous-requête.
- „ Si la sous-requête est vide, **ALL** retourne vrai **ANY** retourne faux,
- „ **SOME** peut être utilisé à la place de ANY.

Exemple 5.22 **ANY / SOME**

„ *Trouver tous les membres du personnel dont le salaire est plus grand qu'au moins un employé de la filiale F003:*

```
SELECT numPersonnel, prénom, nom, fonction, salaire
FROM Personnel
WHERE salaire >SOME (SELECT salaire
                      FROM Personnel
                      WHERE numFiliale = 'F003');
```

„ La requête interne produit l'ensemble {12000, 18000, 24000} et la requête externe sélectionne les employés dont le salaire est plus grand qu'une des trois valeurs de l'ensemble,

Exemple 5.23 ALL

„ Trouver tous les membres du personnel dont le salaire est plus grand que ceux de tous les employés de la filiale F003:

```
SELECT numPersonnel, prénom, nom, fonction, salaire
FROM Personnel
WHERE salaire >ALL (SELECT salaire
                     FROM Personnel
                     WHERE numFiliale = 'F003');
```

numPersonnel	prénom	nom	fonction	salaire
EP21	Jean	Blanc	Gérant	30000.00

REQUÊTES SUR PLUSIEURS TABLES

Requête Multi-Table

- „ On peut utiliser une sous-requête lorsque les colonnes du résultat proviennent d'une seule table,
- „ Si les colonnes du résultat proviennent de plusieurs tables, on doit utiliser une jointure,
- „ Pour faire une jointure, inclure plus d'une table dans le FROM,
- „ Utilisez la virgule comme séparateur et inclure dans le WHERE la ou les colonnes à joindre,
- „ Il est aussi possible d'utiliser un alias pour une table dans le FROM,
- „ Il est séparé du nom de la table par un espace, Il peut être utilisé pour qualifier un nom de colonne (préfixe) là où il y a ambiguïté.

Exemple 5.24 Jointure simple

„ Énumérer les noms et les commentaires de tous les clients qui ont visité une propriété à louer:

```
SELECT c.numClient, prénom, nom, numPropriété,  
commentaire
```

```
FROM Client c, Visite v
```

```
WHERE c.numClient = v.numClient;
```

„ Seules les rangées des deux tables qui ont des valeurs identiques dans la colonne **numClient** (**c.numClient = v.numClient**) sont incluses dans le résultat,

„ Équivalent à **l'équi-jointure** de l'algèbre relationnel.

Formes alternatives de jointure

numClient	prénom	nom	numPropriété	commentaire
CL56	Oscar	Meier	LM36	trop petit
CL56	Oscar	Meier	LG14	
CL56	Oscar	Meier	LM4	
CL62	Marie	Labonté	LG14	pas de salle à manger
CL76	Jean	Martin	LM4	trop éloigné

„ il existe des manières alternatives de spécifier les **jointures**:

FROM Client c **JOIN** Visite v **ON** c.numClient = v.numClient

FROM Client **JOIN** Visite **USING** numClient

FROM Client **NATURAL JOIN** Visite

Exemple 5.25 Trier une jointure

„ Pour chaque filiale, énumérer les numéros et les noms
des membres du personnel qui gèrent des propriétés et
les propriétés qu'ils gèrent:

```
SELECT p.numFiliale, p.numPersonnel, prénom, nom, numPropriété  
FROM Personnel p, PropriétéALouer l  
WHERE p.numPersonnel = l.numPersonnel  
ORDER BY p.numFiliale, p.numPersonnel, numPropriété;
```

numFiliale	numPersonnel	prénom	nom	numPropriété
F003	EM14	David	Renault	LM16
F003	EM37	Anne	Bertrand	LM21
F003	EM37	Anne	Bertrand	LM36
F005	EP41	Julie	Hette	LP94
F007	EG9	Marie	Thale	LG14

Exemple 5.26 jointure de 3 tables

„ Pour chaque filiale, énumérer les numéros et les noms des membres du personnel qui gèrent des propriétés, la ville dans laquelle se trouve la filiale et les propriétés que les employés gèrent:

```
SELECT f.numFiliale, f.ville, p.numPersonnel, prénom,  
nom, numPropriété  
FROM Filiale f, Personnel p, PropriétéALouer l  
WHERE f.numFiliale = p.numFiliale AND  
p.numPersonnel = l.numPersonnel  
ORDER BY f.numFiliale, p.numPersonnel,  
numPropriété;
```

Exemple 5.27 Grouper sur plusieurs colonnes.

„ Trouver le nombre de propriétés gérées par chaque membre du personnel:

```
SELECT p.numFiliale, p.numPersonnel, COUNT(*) AS  
nombre  
FROM Personnel p, PropriétéALouer l  
WHERE p.numPersonnel = l.numPersonnel  
GROUP BY p.numFiliale, p.numPersonnel  
ORDER BY p.numFiliale, p.numPersonnel;
```

numFiliale	numPersonnel	nombre
F003	EM14	1
F003	EM37	2
F005	EP41	1
F007	EG9	1

Computation d'une jointure

„ La procédure pour générer les résultats d'une jointure est:

1. Former le **produit cartésien** de toutes les tables du **FROM**,
2. S'il y a un **WHERE**, appliquer le critère à chaque rangée de la table produite, et ne retenir que les rangées qui satisfont la condition,
3. Pour chaque rangée retenue, déterminer les valeurs de chaque item du **SELECT** pour produire les rangées de la table résultat,
4. Si **DISTINCT** a été spécifié, éliminer les rangées dupliquées de la table résultat.
4. S'il y a un **ORDER BY**, trier la table résultat.

Jointure externe

- „ Si une rangée d'une table jointe n'a pas de correspondance, la rangée est omise de la table résultat,
- „ Les opérations de jointure externe retournent les rangées qui ne satisfont pas la condition de jointure,
- „ Considérons les tables suivantes:

Filiale1	
numFiliale	fVille
F003	Montréal
F004	Bruxelles
F002	Paris

PropriétéALouer1	
numPropriété	lVille
LG14	Genève
LP94	Paris
LM4	Montréal

Jointure externe

„ La jointure (interne) de ces deux tables:

```
SELECT f.*, l.*
```

```
FROM Filiale1 f, PropriétéALouer1 l
```

```
WHERE f.fVille = l.lVille;
```

numFiliale	fville	numPropriété	lville
F003	Montréal	LM4	Montréal
F002	Paris	LP94	Paris

Jointure externe

- „ La table résultat possède deux rangées avec la même ville,
- „ Il n'y a pas de rangées correspondances pour les succursales de Montréal et Paris,
- „ Pour inclure les rangées sans correspondance dans la table résultat, on utilise une jointure externe.

Exemple 5.28 jointure externe gauche

Énumérer toutes les filiales et toute propriété qui se trouve dans la même ville:

```
SELECT f.*, l.*  
FROM Filiale1 f LEFT JOIN PropriétéALouer1 l  
ON f.fVille = l.lVille;
```

Inclut les rangées de la première table (gauche) sans correspondance avec la seconde table (droite),

Les colonnes de la seconde table sont remplies par des NULL,

numFiliale	fville	numPropriété	lVille
F003	Montréal	LM4	Montréal
F004	Bruxelles	NULL	NULL
F002	Paris	LP94	Paris

Exemple 5.29 Jointure externe droite

„ Énumérer toutes les propriétés à louer, ainsi que toute filiale qui se trouve dans la même ville:

```
SELECT f.*, l.*  
  
FROM Filiale1 f RIGHT JOIN PropriétéALouer1 l  
  
ON f.fVille = l.lVille;
```

- „ La jointure externe droite inclut les rangées de la seconde table (droit) qui n'ont pas de correspondance avec les rangées de la première table (gauche),
- „ Les colonnes de la première table correspondante sont remplies de NULL.

Exemple 5.30 jointure externe complète

- „ Énumérer les filiales et les propriétés qui se trouvent dans les mêmes villes, ainsi que les propriétés et les filiales sans correspondantes:

```
SELECT f.*, l.*
```

```
FROM Filiale1 f FULL JOIN PropriétéALouer1 l ON  
f.fVille = l.lVille;
```

- „ Inclut les rangées qui sont sans correspondance dans les deux tables,
- „ Les colonnes sans correspondance sont remplies de NULL,

numFiliale	fville	numPropriété	lVille
NULL	NULL	LG14	Genève
F003	Montréal	LM4	Montréal
F004	Bruxelles	NULL	NULL
F002	Paris	LP94	Paris

EXISTS et NOT EXISTS

- „ EXISTS et NOT EXISTS ne peuvent être utilisés que avec les sous-requêtes,
- „ Vrai si et seulement s'il existe au moins une rangée dans le résultat de la sous-requête,
- „ Faux si la sous-requête retourne une table résultat vide,
- „ NOT EXISTS est l'opposé de EXISTS,
- „ Puisque (NOT) EXISTS ne vérifie que pour l'existence ou la non-existence de rangées dans la sous-requête, la sous-requête peut contenir un nombre quelconque de colonnes,
- „ Il est commun pour les sous-requêtes suivant un (NOT) EXISTS d'être de la forme:(SELECT*).

Exemple 5.31 EXISTS

„ Trouver tous les membres du personnel qui travaillent à
la filiale de Paris:

```
SELECT numPersonnel, prénom, nom, fonction  
FROM Personnel p  
WHERE EXISTS (SELECT *  
FROM Filiale f WHERE p.numFiliale =  
f.numFiliale AND ville = 'Paris');
```

numPersonnel	prénom	nom	fonction
EP21	Jean	Blanc	Gérant
EP41	Julie	Hette	Assistant

Union, Intersect et Difference

(Except)

- „ On peut utiliser les opérations normales sur les ensemble (Union, Intersection et Difference) pour combiner les résultats de deux ou plusieurs requêtes dans une seule table résultat,
- „ l'Union de deux tables, A et B, donne toutes les rangées provenant soit de A, soit de B ou les deux,
- „ l'intersection donne toutes les rangées communes de A et B,
- „ la différence donne toutes les rangées de A qui ne sont pas dans B,
- „ Les deux tables étant compatible envers l'union.

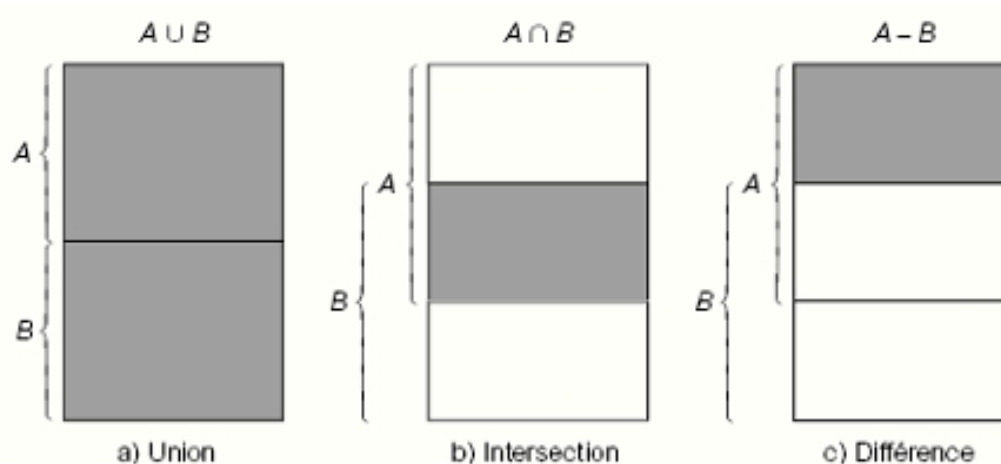
Union, Intersect et Difference

(Except)

„ Le format d'un opérateur d'ensemble dans chaque cas est:

Requête1 op [ALL] Requête2

„ Si ALL est spécifié, le résultat peut inclure des rangées dupliquées,



Exemple 5.32 UNION

„ Édifier une liste de toutes les villes qui comptent une filiale et (ou) une propriété à louer:

```
(SELECT ville FROM Filiale WHERE ville IS NOT NULL)
```

UNION

```
(SELECT ville FROM PropriétéALouer WHERE ville IS  
NOT NULL);
```

ville
Paris
Montréal
Genève
Bruxelles

Exemple 5.33 INTERSECT

„ Édifier une liste de toutes les villes qui comptent à la fois
une filiale et au moins une propriété à louer:

(SELECT ville FROM Filiale)

INTERSECT

(SELECT ville FROM PropriétéALouer);

ville
Genève
Montréal
Paris

Exemple 5.33 INTERSECT

„ On peut réécrire cette requête INTERSECT :

```
SELECT ville FROM Filiale f, PropriétéALouer l  
WHERE f.ville = l.ville;
```

„ Ou:

```
SELECT DISTINCT ville FROM Filiale f  
WHERE EXISTS (SELECT * FROM PropriétéALouer l  
WHERE f.ville = l.ville);
```

Exemple 5.34 EXCEPT

„ Édifier une liste de toutes les villes qui comptent une filiale mais aucune propriété à louer:

```
(SELECT ville FROM Filiale)
```

```
EXCEPT
```

```
(SELECT ville FROM PropriétéALouer);
```

„ On peut réécrire cette requête sans EXCEPT:

```
SELECT DISTINCT ville FROM Filiale
```

```
WHERE ville NOT IN (SELECT ville FROM  
PropriétéALouer);
```

ville
Bruxelles

Vues

Définition :

- „ Résultat dynamique de une ou plusieurs opérations relationnelles s'effectuant sur les relations de base pour produire une autre relation,
- „ Une relation virtuelle qui n'existe pas dans la base de donnée mais qui est reproduite sur demande, à chaque demande,

Remarques :

- „ Le contenu d'une vue est définit comme une requête sur une ou plusieurs relations de base,
- „ Avec la résolution de vue, toutes les opérations sur une vue sont automatiquement traduites en opérations sur les relations dont elles sont dérivées,
- „ Avec la résolution de vue, la vue est emmagasinée comme une table temporaire, laquelle est maintenue telle que les tables sous-jacentes sont mises à jour.

SQL - CREATE VIEW

```
CREATE [MATERIALIZED] VIEW NomVue [  
    (NouveauNomColonne[,...]) ]  
    AS sousSelect  
    [WITH CHECK OPTION]
```

- „ On peut assigner un nouveau nom à chaque colonne de la vue,
- „ Si la liste de noms de colonne est spécifiée, il doit y avoir le même nombre d'items que le nombre de colonne produite par le SELECT,
- „ Si omise, chaque colonne prend le nom de la colonne correspondante du select.

SQL - CREATE VIEW

- „ La liste doit être spécifiée s'il y a une ambiguïté dans les noms de colonne,
- „ Le select est nommé requête de définition,
- „ WITH CHECK OPTION assure que si une rangée ne peut satisfaire la clause WHERE de la requête de définition, elle n'est pas ajoutée à la table de base sous-jacente.

Exemple 6.3 - Créer une vue horizontale

- „ Une *vue horizontale* est une vue qui restreint l'accès de l'utilisateur à certaines ligne d'une ou plusieurs tables,
- „ *Créer une vue telle, que le gérant de la filiale F003 puisse voir les seuls détails des membres du personnel qui travaillent dans sa filiale:*

```
CREATE VIEW Gérant3Personnel AS SELECT *  
FROM Personnel WHERE numFiliale = 'F003' ;
```

numPersonnel	prénom	nom	fonction	genre	dateDeNaissance	salaire	numFiliale
EM37	Anne	Bertrand	Assistant	F	10 Nov 60	12000.00	F003
EM14	David	Renault	Superviseur	M	24 Mar 58	18000.00	F003
EM5	Suzanne	Brière	Gérant	F	3 Jun 40	24000.00	F003

Exemple 6.4 - Créer une vue verticale

- „ Une vue verticale est une vue qui restreint l'accès de l'utilisateur à certaines colonnes d'une ou plusieurs tables,
- „ *Créer une vue des détails des membres du personnel de la filiale F003 qui exclut les informations de salaire, de façon que seuls les gérants puissent accéder aux détails de salaire des employés qui travaillent dans leur filiale:*

```
CREATE VIEW Personnel3 AS SELECT  
    numPersonnel, prénom, nom, fonction, genre FROM  
    Personnel WHERE numfiliale = 'F003' ;
```

numPersonnel	prénom	nom	fonction	genre
EM37	Anne	Bertrand	Assistant	F
EM14	David	Renault	Superviseur	M
EM5	Suzanne	Brière	Gérant	F

Exemple 6.5 - Vues groupées et jointes

„ Créer une vue du personnel qui gère des propriétés à louer, reprenant le numéro de la filiale où l'employé travaille, son numéro d'employé et le nombre de propriétés qu'il gère:

```
CREATE VIEW PersonnelCompteProp (numFiliale,  
    numPersonnel, nombre) AS SELECT p.numfiliale,  
    p.numPersonnel, COUNT(*) FROM Personnel p,  
    PropriétéALouer l WHERE p.numPersonnel =  
    l.numPersonnel GROUP BY p.numfiliale,  
    p.numPersonnel ;
```

numFiliale	numPersonnel	nombre
F003	EM14	1
F003	EM37	2
F005	EP41	1
F007	EG9	1

SQL - DROP VIEW

`DROP VIEW nomVue [RESTRICT | CASCADE]`

„ La définition de la vue est effacée de la base de données,

„ Par exemple:

`DROP VIEW Gérant3Personnel;`

„ Avec CASCADE, tous les objets dépendant sont effacés; c'est-à-dire que les vues définies sur la vue sont effacées,

„ Avec RESTRICT (par défaut), si tout autre objet dépend de la vue (et non l'inverse), la commande est rejetée.

Résolution de vue

- „ Compter le nombre de propriétés à louer administrées par chaque membre du personnel de la filiale F003:

```
SELECT numPersonnel, nombre FROM  
    PersonnelCompteProp WHERE numfiliale = 'F003'  
ORDER BY numPersonnel ;
```

1. Les noms de colonnes de la vue dans le SELECT sont traduit en leur nom de colonne correspondant de la requête de définition de la vue:

```
SELECT p.numPersonnel AS numPersonnel,  
    COUNT(*) AS nombre
```

Résolution de vue

2. Les noms de vue dans le FROM sont remplacés par les noms correspondant du FROM de la requête de définition de la vue:

FROM Personnel p, PropriétéALouer l

3. WHERE de la requête de l'utilisateur est combiné avec le WHERE de la requête de définition de la vue avec AND:

**WHERE p.numPersonnel = l.numPersonnel AND
numfiliale = 'F003'**

4. GROUP BY et HAVING sont copiés à partir de la requête de définition:

GROUP BY p.numfiliale, p.numPersonnel

Résolution de vue

5. ORDER BY est copié de la requête avec les noms de colonne de la vue traduits dans les noms de colonnes de la requête de définition

ORDER BY p.numPersonnel

6. La requête finale fusionnée est alors exécutée pour produire le résultat:

**SELECT p.numPersonnel AS numPersonnel,
COUNT(*) AS nombre FROM Personnel p,
PropriétéALouer l WHERE p.numPersonnel =
l.numPersonnel AND numfiliale = 'F003' GROUP BY
p.numfiliale, p.numPersonnel ORDER BY
p.numPersonnel ;**

Restrictions sur les vues

- „ SQL impose plusieurs restrictions sur la création et l'utilisation des vues,
- „ (a) Si une colonne de la vue est basée sur une fonction D'agrégation
 - ... La colonne peut apparaître seulement dans un **SELECT** ou **ORDER BY** de la requête qui accède à la vue,
 - ... La colonne ne peut être utilisée dans le **WHERE** ni dans une fonction d'agrégation de la requête qui appelle la vue.

Restrictions sur les vues

„ Par exemple, la requête suivante échouerait:

```
SELECT COUNT(nombre) FROM  
PersonnelCompteProp ;
```

„ De façon similaire, la requête suivante échouerait aussi:

```
SELECT * FROM PersonnelCompteProp  
WHERE  
    nombre > 2 ;
```

„ (b) Une vue groupée ne pourra jamais être jointe à une autre vue ou une table de base,

„ Par exemple, la vue PersonnelCompteProp est une vue groupée, donc toute tentative de jointure sur cette vue avec une table ou une autre vue échoue.

Aspect modifiable d'une Vue

- „ Toutes les mise à jour dans les tables de base se répercutent dans toutes les vues qui utilisent ces tables,
- „ De façon similaire, on peut espérer que si la vue est Mise à jour les tables de base reflètent ces changements,
- „ Si nous essayons d'insérer les enregistrements démontrant qu'à la succursale F003, EM5 gérant 2 propriétés:

```
INSERT INTO PersonnelCompteProp  
VALUES ('F003', 'EM5', 2) ;
```


Aspect modifiable d'une Vue

- „ Nous devons insérer 2 enregistrements dans PropriétéÀLouer présentant quelles propriétés EM5 gère. Cependant, on ne sait quelle propriété; c'est-à-dire on ne connaît pas les clés primaires,
- „ Si on change la définition de la vue et que l'on remplace le count par les numéros de propriété:

```
CREATE VIEW PersonnelCompteProp (numFiliale,  
    numPersonnel, numPropriété)  
AS SELECT p.numfiliale, p.numPersonnel,  
    l.numPropriété  
FROM Personnel p, PropriétéÀLouer l  
WHERE p.numPersonnel = l.numPersonnel ;
```

Aspect modifiable d'une Vue

„ Maintenant, essayons d'insérer l'enregistrement:

```
INSERT INTO ListPersonnelPropriété  
VALUES ('F003', 'EM5', 'LM19');
```

- „ Un problème reste, puisque dans PropriétéÀLouer toutes les colonnes sauf codePostal/numPersonnel ne permettent pas les NULL,
- „ Cependant, nous n'avons pas de moyens de fournir des valeurs pour les valeurs de colonne non-null restantes.

Aspect modifiable d'une Vue

- „ ISO spécifie que les vues doivent être modifiable dans les systèmes qui se conforment au standard,
- „ Une vue est modifiable si et seulement si:
 - ... **DISTINCT** n'est pas spécifié,
 - ... Chaque élément du **SELECT** de la requête de définition est un nom de colonne et aucune colonne n'apparaît plus d'une fois,
 - ... Le **FROM** ne spécifie qu'une seule table, excluant ainsi toute vue basée sur une jointure, une union, une intersection ou une différence,
 - ... Aucun sous-select référénçant une table externe.

Aspect modifiable d'une Vue

... Aucun **GROUP BY** ou **HAVING**,

... De plus, chaque rangée ajoutée au travers d'une vue ne doit pas violer les contraintes d'intégrités de la table de base,

„ Pour qu'une vue soit modifiable, le SGBD doit être capable de retracer toute rangée ou colonne dans sa table d'origine.

WITH CHECK OPTION

- „ Les rangées existent dans la vue car elles satisfont le **WHERE** de la requête de définition,
- „ Si une rangée change et ne satisfait plus la condition, elle disparaît de la vue,
- „ Les nouvelles rangées qui apparaissent dans la vue lors de insert/update sur la vue leur permet de satisfaire le **WHERE**,
- „ Les rangées qui entrent et sortent de la vue sont appelés **rangées migrantes**,
- „ **WITH CHECK OPTION** empêche à une rangée de migrer hors de la vue.

Exemple 6.6 - WITH CHECK OPTION

```
CREATE VIEW Gérant3Personnel  
AS SELECT *  
FROM Personnel  
WHERE numfiliale = 'F003'  
WITH CHECK OPTION;
```

- „ On ne peut mettre à jour le numéro de succursale de la rangée F003 à F002 car cela ferait migrer la rangée hors de la vue,
- „ De plus, on ne peut insérer une rangée dans la vue avec un numéro de succursale différent de F003.

Avantages des vues

- „ Indépendance de donnée,
- „ Précision,
- „ Sécurité améliorée,
- „ Réduit la complexité,
- „ Convivialité,
- „ Personnalisation,
- „ Intégrité des données,

„ Inconvénients:

- ... Restriction de mise à jour,
- ... Restriction de structure,
- ... Performance.

Matérialisation de vue

- „ Le mécanisme de résolution de vue peut être lent, particulièrement si la vue est accédée fréquemment.
- „ La matérialisation de vue permet d'emmagasinier la vue dans une table temporaire lorsque la vue est appelée pour la première fois.
- „ Ensuite, les requêtes basées sur la vue matérialisée peuvent être plus rapide que si elle sont recalculées à chaque fois.
- „ La difficulté est de maintenir la précision de la vue lorsque la table de base est mise à jour.

Maintenance de vue

- „ La maintenance de vue permet d'appliquer seulement les changements nécessaires pour garder la vue à jour,
- „ Considérons la vue suivante:

```
CREATE VIEW PersonnelLocPAL  
AS SELECT DISTINCT numPersonnel  
FROM PropriétéALouer  
WHERE numFiliale = 'F003'  
AND location > 400 ;
```

Tableau 6.8
Données de la vue
PersonnelLocPAL.

numPersonnel
EM37
EM14

Matérialisation de vue

- „ Si on insert une rangée dans **PropriétéÀLouer** avec un paiement ≤ 400 alors la vue reste inchangé.
- „ Si on insert la rangée pour la propriété LM24 de la succursale F003 avec numPersonnel = EM19 et le paiement = 550, alors la rangée apparaîtra dans la vue matérialisée,
- „ Si on insert la rangée de la propriété LM54 de la succursale F003 avec numPersonnel = EM37 et le paiement = 450, alors aucune nouvelle rangée n'aura besoin d'être ajouté à la vue matérialisée,
- „ Si on efface la propriété LM24, la rangée doit être efficace de la vue matérialisée,
- „ Si on efface la propriété LM54, alors la rangée pour LM37 ne devra pas être effacée (à cause de l'existence la propriété LM21).

Références :

- Livres de A.Gamache
- Support de M.P.Parent